

Design Justification Report

Ray-Object Intersection Accelerator Chiplet

ECE 510 HW4AI Spring 2026 — Milestone 4

Author: Rajeshwar Vemula | Date: June 2026

PDK: sky130_fd_sc_hd | Clock: 40 MHz (25 ns) | Format: Q8.8 Fixed-Point

1. Problem and Motivation

Ray-object intersection testing is the dominant computational kernel in ray tracing, consuming over 30% of total execution time in scene rendering. In our M1 profiling of a Python ray tracer rendering a 14-object city scene at 400x300 resolution with depth-5 bounces, cProfile measured 419.3 seconds for 30 frames. The three intersection kernels accounted for 30.4% of total runtime: `intersect_box` (264s, 39M calls, 17.5%), `intersect_sphere` (97s, 66M calls, 6.4%), and `intersect_cylinder` (97s, 39.7M calls, 6.5%). The total intersection throughput was 503,000 tests per second, limited by Python interpreter overhead.

For embedded ADAS applications, LiDAR simulation requires real-time intersection testing at millions of rays per second. The Python baseline achieves only 0.072 fps — four orders of magnitude below the 30 fps required for real-time operation. A custom hardware accelerator eliminates interpreter overhead and exploits the fixed arithmetic structure of the quadratic intersection formula to deliver deterministic, low-latency results at a fraction of the energy cost of a general-purpose processor.

This project implements a fixed-function ray-sphere intersection pipeline as a proof of concept. The sphere intersection kernel was selected because it is the most computationally intensive per call (quadratic formula with square root) and exercises all arithmetic operations needed for other intersection types.

2. Roofline Analysis

The ray-sphere intersection kernel performs 39 floating-point equivalent operations per test: three dot products (5 FLOPs each = 15), discriminant calculation (5 FLOPs), square root (approximately 16 FLOPs for 16 iterations), and final t calculation (3 FLOPs). Each test requires 24 bytes of input (10 Q8.8 parameters) and produces 4 bytes of output. The arithmetic intensity without register reuse is $AI = 39/24 = 1.62$ FLOP/byte.

On the reference Cortex-M7 platform (400 MFLOP/s peak, 3.2 GB/s bandwidth, ridge at 0.125 FLOP/byte), the kernel is compute-bound: $AI = 1.62$ exceeds the ridge. The software baseline achieves 19.5 MFLOP/s — 5% of peak — due to Python overhead.

The chiplet at 40 MHz with 40 MB/s AXI bandwidth has a ridge point at 1.625 FLOP/byte. With register reuse (sphere parameters stationary, ray direction streamed), the effective AI rises to 4.88 FLOP/byte — firmly compute-bound. The measured 65 MFLOP/s sits at the chiplet compute ceiling, confirming the design is not bandwidth-limited. See Figure 1 (roofline_final.png).

3. Precision and Data Format

The design uses Q8.8 signed fixed-point (16-bit): 8 integer bits (including sign) and 8 fractional bits, with 1 LSB = 1/256. This format was chosen over FP32 for three reasons: (1) halving data width cuts AXI traffic by 2x, (2) integer multipliers are significantly smaller than FP32 units on sky130 (no exponent alignment), and (3) the precision is sufficient for the target application.

Intermediate products use Q16.16 (32-bit) and the discriminant uses Q32.32 (64-bit). The square root input is disc[47:16] (Q16.16 portion), and the non-restoring sqrt produces Q8.8 directly because $\text{sqrt}(N * 2^{16}) = \text{sqrt}(N) * 2^8$.

Error analysis (M2 precision.md) measured worst-case error of 5 LSB = 0.0195 scene units across 22 test vectors, with mean 1.75 LSB. This is within the 0.025 pixel threshold at 400x300 and within the 2-3 cm LiDAR noise floor. The 999-test randomised suite confirmed 999/999 exact match between hardware and Q8.8 software model, proving all error is inherent to the format.

4. Dataflow and Architecture

The compute_core implements an input-stationary dataflow. Sphere parameters (center + radius = 8 bytes) are loaded into AXI4-Lite registers and held stationary while ray directions stream through AXI4-Stream. This matches the ray tracing access pattern: for each pixel, the same set of objects is tested against a new ray. Sphere parameters change only when the host reconfigures for the next object.

The pipeline FSM has 9 states executing in 24 cycles: IDLE (input latch), STAGE_OS (OS = O - S), STAGE_DOTS (three dot products), STAGE_MUL (b*b and a*c in parallel — added in M4 to fix critical path), STAGE_DISC (disc = b_sq - 4*ac via shift+subtract), SQRT_INIT (sign check), SQRT_RUN (16 iterations), STAGE_T (combinational t), and DONE_STATE (output latch).

The memory hierarchy is minimal: all operands held in 479 flip-flop registers. No SRAM, no cache, no scratchpad. This makes the pipeline fully deterministic — every test completes in exactly 24 cycles regardless of input. The input-stationary pattern saves 8 bytes per test when testing multiple rays against the same sphere (333 rays per sphere saves 2,664 bytes of reloaded data).

5. Hardware Interface

The interface uses AXI4-Lite for configuration (sphere params at 0x1C-0x28, ray origin at 0x04-0x0C) and AXI4-Stream for data streaming (ray direction in s_axis_tdata[47:0], result in m_axis_tdata[15:0]). The stream handshake provides backpressure: s_axis_tready deasserts when the pipeline is busy, preventing input overwrite during computation.

Effective bandwidth at 40 MHz: in streaming mode, 8 bytes per test every 24 cycles = 13.3 MB/s input. When time-multiplexing objects, the AXI4-Lite reload adds ~28 cycles per object switch. The design is compute-bound, not interface-bound: the 24-cycle pipeline dominates over AXI overhead.

6. Verification

Verification proceeded in three stages. M2: `tb_compute_core.sv` (4 tests, 8/8 PASS) tested the isolated pipeline; `tb_interface.sv` (13/13 PASS) tested AXI handshakes. M3: `tb_top.sv` drove the integrated top module through AXI only — 22 hand-picked vectors across 3 spheres (13 hits + 9 misses, 22/22 PASS with 3 LSB tolerance), plus 999 randomised tests (999/999 PASS at 0 tolerance). M4: same 22-test suite re-run after `STAGE_MUL` pipeline split, 22/22 PASS confirming functional equivalence.

7. Synthesis Results

Metric	M3 (20 ns)	M4 (25 ns)
Total cells	41,118	41,367
Cell area	248,852 um2	257,104 um2
Utilization	35.9%	35.6%
Setup WNS (slow)	-2.314 ns FAIL	+2.032 ns PASS
Setup WNS (typical)	+8.667 ns	+13.702 ns
Hold violations	0	0
Power	33.75 mW	39.6 mW
DRC / LVS	0 / 0	0 / 0

The M4 pipeline split (`STAGE_MUL`) improved slow-corner WNS from -2.314 ns to -1.628 ns at 20 ns — proving the discriminant was the critical path. However, a single 32x32 multiply inherently takes ~22 ns on sky130 HD at the slow corner. Retargeting to 25 ns (40 MHz) achieved clean timing closure: +2.032 ns slack at the worst corner, 0 setup violations, 0 hold violations across all 9 PVT corners.

The dominant area contributor is combinational logic (96.4%): XNOR2 (3,286 instances), NAND2 (2,691), and NOR2 (2,148) form multiplier partial products and carry chains. Sequential elements (479 FFs) account for 3.6%.

8. Benchmark Results

Metric	SW Baseline	HW Accel	Ratio
Tests/sec	500,835	1,666,667	3.3x
MFLOP/s	19.5	65.0	3.3x
Power	15 W (desktop)	39.6 mW	379x less
Energy/test	29.9 uJ	23.8 nJ	1,257x less
Energy/frame	209.6 J	39.9 mJ	5,254x less

The 3.3x throughput speedup is conservative: the SW baseline includes Python interpreter overhead. The energy advantage is the primary value proposition — 23.8 nJ per test is 1,257x more efficient than a desktop CPU. For ADAS systems processing millions of tests per second, this maps directly to battery life and thermal budget.

The gap between measured (65 MFLOP/s) and M1 target (900 MFLOP/s) is due to: (1) single pipeline instead of 14 parallel (14x), (2) 40 MHz instead of 50 MHz (1.25x), (3) 24-cycle FSM instead of fully-pipelined 1-cycle throughput (24x). By Amdahl's law, accelerating only `intersect_sphere` (6.4% of

runtime) yields 1.047x whole-app speedup. Extending to all intersection types would cover 30%.

9. What Did Not Work

Parallel pipeline scope reduction. M1 planned 14 parallel intersection pipelines. M3 synthesis revealed one pipeline = 248,852 μm^2 ; 14 parallel = 3.5M μm^2 — infeasible. The M1 estimate of 640 cells was 64x too low because it did not account for synthesized multiplier expansion. Lesson: run synthesis before M2, not after M3.

Slow-corner timing at 50 MHz. The discriminant stage (two 32x32 multiplies) had a 22.3 ns critical path, violating 20 ns by 2.31 ns. M4 pipelined it into STAGE_MUL + STAGE_DISC, improving to -1.63 ns. A single 32x32 multiply still exceeds 20 ns on sky130 HD slow corner (~21.6 ns). Retargeted to 25 ns (40 MHz) for clean closure (+2.03 ns slack). The sky130_fd_sc_hs (high-speed) library was not available in the Docker PDK.

Icarus Verilog limitations. The AXI4-Lite testbench hung in Icarus 12.0 due to unsupported SystemVerilog constructs. Three handshake implementations were attempted before switching to VCS. Lesson: use production EDA tools from the start.

VCD waveform generation. \$dumpfile/\$dumpvars and +vcs+vcdpluson both failed to produce waveform files on the school VCS server. A matplotlib timing diagram was used as a substitute.

Q8.8 precision tradeoff. Switching from FP32 to Q8.8 reduced area by ~60% but introduced up to 5 LSB error (0.0195 scene units) at longer distances. Acceptable for ADAS but insufficient for sub-pixel anti-aliasing. Q12.12 would be the minimum for that application.

Git force-push data loss. A force-push overwrote codefest 5 and 6 submissions on GitHub. They were remote-only and could not be recovered. Lesson: never use --force on shared branches.